

Software Engineering II

Domain-Driven-Design

Mathis Neunzig
ABAP Developer at SAP
dhbw@mathis-neunzig.de



Architekturmuster

Was wir heute in der VL machen
ist grün markiert!

Strukturmuster

- Layered Architecture
- MVC / MVP / MVVM
- Pipe-and-Filter
- Microkernel (Plug-in Architecture)
- Modular Monolith
- Component-Based Architecture
- Self-contained Systems (SCS)
- Blackboard Architecture

Verteilungsmuster

- Client-Server
- Microservices
- Service-Oriented Architecture (SOA)
- Peer-to-peer (P2P)
- Event-Driven Architecture (EDA)
- Space-Based Architecture
- Shared Nothing Architecture
- Service Mesh

Verantwortlichkeits-trennungsmuster

- Domain-Driven-Design (DDD)
- Command Query Responsibility Segregation (CQRS)
- Hexagonal Architecture
- Onion Architecture
- Clean Architecture
- Screaming Architecture

Deployment- & Betriebsmodelle

- Serverless Architecture
- Containerized Architecture
- Backend for Frontend
- API Gateway Pattern
- Sidecar Pattern
- Multi-Tenant Architecture
- Twelve-Factor App
- Immutable Infrastructure

Integrationsmuster

- Adapter
- Facade
- Message Bus & Message Queue
- Publish-Subscribe
- Saga Pattern
- Anti-Corruption Layer
- Gateway
- Event-Carried State Transfer
- Shared Database

Kommunikationsmuster (nicht offiziell Teil von Architekturmustern): REST, GraphQL, SOAP, Odata, Websocket, Remote Procedure Call, Graph-based API, JSON-RPC, XML-RPC

Domain-Driven-Design

- Ansatz zur Entwicklung, Domäne steht im Mittelpunkt
- Eric Evans (2003), Buch „Domain-Driven Design“
- Ziel: Modell = Code = gemeinsame Sprache
- Zwei Ebenen:
 - Strategisches Design (Kontextabgrenzung)
 - Taktisches Design (Bausteine der Umsetzung)
- Vorteil: weniger Missverständnisse, langlebige Software

Context

Customer Support

Train Ticket Booking

Context

Customer Support

Raise

Ticket

Login

Account

Train Ticket Booking

Kontext

Customer Support

Raise

Ticket

Login

Account

Train Ticket Booking

Buy

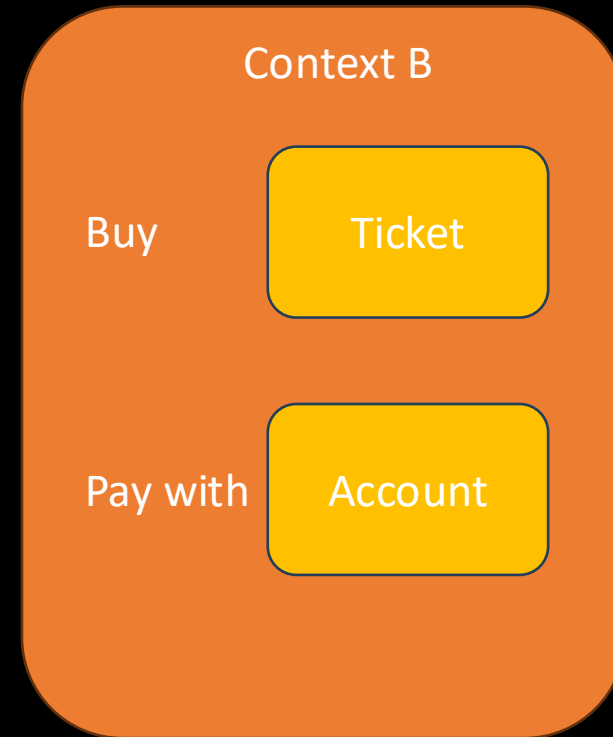
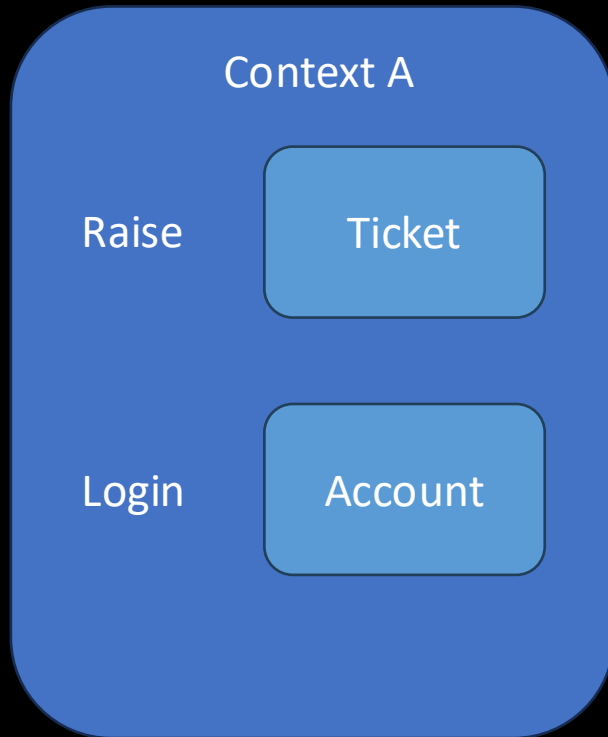
Ticket

Pay with

Account

Bounded Context

- “A part of the software where particular terms, rules and definitions apply consistently”



Ubiquitous Language

- Reduziert Übersetzungsaufwand, Fach- und Entwicklersicht synchron
- Sprache im engen Austausch mit Domänenexperten entwickeln
- Glossar erstellen & pflegen für alle wichtigen Begriffe
- Beispiel: „Auftrag“ in Code & Fachbereich identisch
- Zuhören, hinterfragen, Fachbegriffe genau aufnehmen
- Kein Übersetzen in „Programmiererbegriffe“ nötig

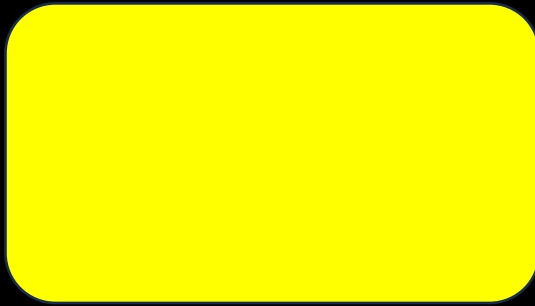
Strategisches Design

- Bounded Context: klar abgegrenzter Bereich, eigene Sprache
- Beispiel: „Auftrag“ im Bestell- vs. Lagerkontext
- Context Map: Visualisierung der Beziehungen der Kontexte
- Muster:
 - Partnership: enge Zusammenarbeit
 - Customer/Supplier: Zulieferer-Kunde
 - Conformist: Übernahme von Konzepten
 - Anticorruption Layer: Schutz vor fremden Modellen
- Ziel: Konflikte und Missverständnisse vermeiden

Taktisches Design

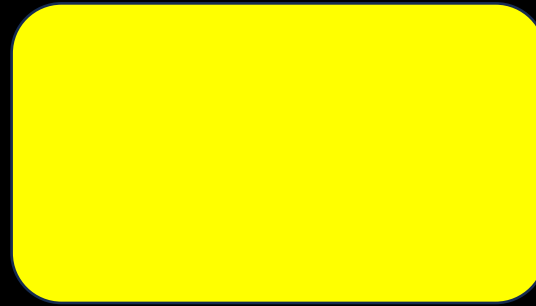
- Entities: mit Identität, veränderlich (z. B. Kunde)
- Value Objects: keine Identität, reine Werte (z. B. Adresse)
- Aggregates: Konsistenzgrenzen, Root-Entity als Einstieg (& Regeln)
- Events: Events, die eine Entity verändern
- Repositories: Zugriff & Speicherung abstrahieren
- Services: Fachlogik außerhalb von Entities
- Factories: komplexe Erstellung von Aggregaten kapseln

Value Objects



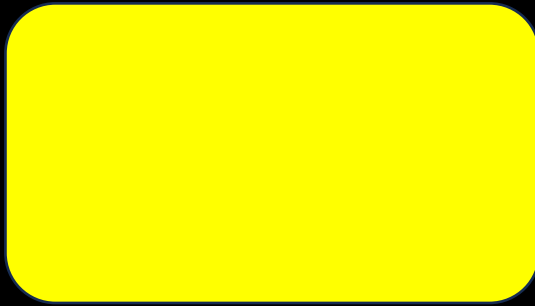
R = 255
G = 250
B = 5

==?



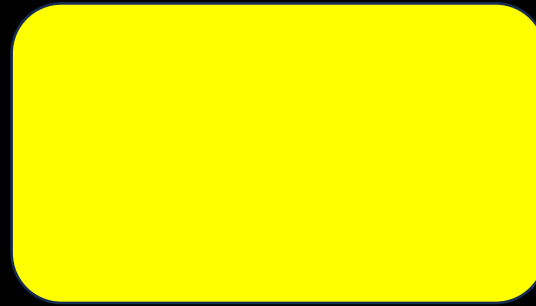
R = 255
G = 250
B = 5

Value Objects



R = 255
G = 250
B = 5

=



R = 255
G = 250
B = 5

Entity

Id = 1



Name = Jack
Color = Cat
Eyes = Brown

==?

Id = 2



Name = Jack
Color = Cat
Eyes = Brown

Entity

Id = 1



Name = Jack
Color = Cat
Eyes = Brown

!=

Id = 2



Name = Jack
Color = Cat
Eyes = Brown

Entity, Value Object oder Aggregate

- Person
- Product
- Order
- Color
- Bill of Material
- Address

Entity, Value Object oder Aggregate

- Person → Entity
- Product
- Order
- Color
- Bill of Material
- Address

Entity, Value Object oder Aggregate

- Person → Entity
- Product → Entity (oder in bestimmten Fällen auch Aggregate)
- Order
- Color
- Bill of Material
- Address

Entity, Value Object oder Aggregate

- Person → Entity
- Product → Entity (oder in bestimmten Fällen auch Aggregate)
- Order → Aggregate
- Color
- Bill of Material
- Address

Entity, Value Object oder Aggregate

- Person → Entity
- Product → Entity (oder in bestimmten Fällen auch Aggregate)
- Order → Aggregate
- Color → Value Object
- Bill of Material
- Address

Entity, Value Object oder Aggregate

- Person → Entity
- Product → Entity (oder in bestimmten Fällen auch Aggregate)
- Order → Aggregate
- Color → Value Object
- Bill of Material → Aggregate
- Address

Entity, Value Object oder Aggregate

- Person → Entity
- Product → Entity (oder in bestimmten Fällen auch Aggregate)
- Order → Aggregate
- Color → Value Object
- Bill of Material → Aggregate
- Address → Entity oder Value Object (abhängig von der Wichtigkeit)

Danke!

Mathis Neunzig
dhbw@mathis-neunzig.de

